

Glassbox for IT

What it reads, what it writes, what it runs, what it sends. Written for the person who has to approve it, checked against the source of glassbox-trace 0.6.2. If anything here stops being true, that is a bug — open an issue.

Glassbox is a viewer and a checker for the transcript files Claude Code already writes to every developer's machine. It has no server, no account, no telemetry and no runtime dependencies. Everything below runs as the user, on the user's machine.

1. The three ways it can be on a machine

	what lands on the machine	how it gets there	how it runs
The HTML file	one file, <code>glassbox.html</code> (595 KB; fonts, code and demo inside)	download from GitHub Releases / npm, or copy from an internal share	double-click; runs in the browser from <code>file://</code>
The CLI	npm package <code>glassbox-trace</code> : 9 files, ~150 KB packed, zero dependencies (package.json has no dependencies field); published with npm provenance from the GitHub Actions workflow in this repo	<code>npx glassbox-trace</code> (fetches on each run unless cached) or <code>npm i -g glassbox-trace@0.6.2</code> (fetches once, pinned)	<code>node ≥ 18</code>
The Claude Code plugin	a clone of this repository under Claude Code's plugin directory	<code>/plugin marketplace add aldohush1-stack/glassbox</code> then <code>/plugin install glassbox@glassbox-trace</code>	<code>node</code> running the bundled copy; no npx, no network after install

For a managed fleet the simplest shape is: the HTML file on an internal share (nothing to install, nothing to update automatically) plus, where the CLI is wanted, a global install of a pinned version.

2. Network

The viewer makes no network requests. Fonts, scripts and the demo session are inside the file. This is enforced by the test suite: `test/offline.test.mjs` fails the build if the file references anything that is not a `data:` URL, and the browser test logs every request the page makes and fails if any is not `file:`, `data:` or `blob:`. (Versions up to 0.6.1 loaded one stylesheet from `fonts.googleapis.com`; that is gone.)

The CLI makes no network requests. `list`, `open`, `check`, `compare`, `reveal`, `hook` and `watch` open no sockets to anything outside the machine.

`glassbox watch` starts an HTTP server bound to `127.0.0.1` on a random free port (or `--port N`) for the lifetime of the command, to stream the transcript to the browser tab it opens. It is not reachable from other machines and stops with `Ctrl+C`.

The only network activity is installation: npm fetching the package (`npx` does this on each run if the package is not cached; a global install does it once), or Claude Code's plugin marketplace cloning this repository.

3. Files it reads

- `~/.claude/projects/<project>/<session-id>.jsonl` and the `<session-id>/subagents/**` beside it — Claude Code's own transcripts. `GLASSBOX_HOME` or `--home` points at a different `.claude` directory. `list` reads directory entries and file sizes; `list --grep` reads every transcript's full text to search it; `open`, `check`, `compare`, `watch` read the session(s) named.
- A `.jsonl` file or folder the user names on the command line or drops onto the page.
- `~/.claude/settings.json` — only by `hook install` / `hook uninstall`, and by the plugin `hook` to see whether a CLI hook is already installed.
- `--rates FILE` / `GLASSBOX_RATES` (a rate card), `--legend FILE` (a legend), when given.
- In the browser, **Open folder...** uses the File System Access API (Chrome/Edge): the user picks a folder, the browser asks permission, and the page reads `.jsonl` files under it. The permission is per site and per folder, revocable in the browser.

These transcripts contain everything the agent saw: prompts, file contents, command output. Glassbox does not change that fact; it reads what is already on disk.

4. Files it writes

command	writes	contains	lifetime
<code>glassbox open <id></code>	<code><temp>/glassbox-<id8>.html</code> (<code>%TEMP%</code> on Windows, <code>\$TMPDIR</code> elsewhere)	the viewer with the transcript embedded	not deleted automatically. Use <code>--out PATH</code> to choose the location, <code>--no-open</code> to skip the browser, and delete the file when done
<code>glassbox open <id> -out F, compare A B --out F</code>	the file named	viewer + transcript(s)	until deleted
<code>glassbox check --legend F</code>	<code>F</code>	salt + key → path map for every file path in the report	until deleted; keep it local, it is the key to the redacted report
<code>glassbox hook install</code>	<code>~/.claude/settings.json</code> (+ <code>settings.json.glassbox-backup</code>)	a <code>Stop</code> (and with <code>--context</code> , <code>SessionStart</code>) hook entry	until <code>hook uninstall</code>
Stop hook with <code>--feedback</code> / plugin option <code>feedback</code>	<code><temp>/glassbox-hook/<session-id>.json</code>	ids and titles of findings already handed back (counts and rule names; a title can name a tool or, for <code>duplicate-subagent-read</code> , a file path)	not deleted automatically

command	writes	contains	lifetime
Stop hook with <code>--context / plugin option context</code>	<code><project>/ .glassbox/last-session.md</code> and <code><project>/ .glassbox/.gitignore</code>	the findings report for that session ($\leq 6,000$ chars; quotes tool inputs and outputs the way <code>check --format md</code> does) and, with feedback on, the agent's one-line reply	deleted by the hook after a clean session; <code>.gitignore</code> keeps it out of git
viewer Export report / Export redacted	a download the user saves	Markdown findings / structure-only <code>.jsonl</code>	user's choice

Nothing is written anywhere else. There is no config file, no cache and no state outside the rows above.

5. Processes it starts

- `open` and `watch` open the browser through the platform opener: `cmd /c start "<file>"` on Windows, `open` on macOS, `xdg-open` on Linux, or whatever `GLASSBOX_BROWSER` names. `--no-open` suppresses it.
- The Stop hook and SessionStart hook run `node` once per event (see §6).
- Nothing else: no daemons, no scheduled tasks, no services, no shell-outs to anything but the opener above.

6. The hooks, exactly

Claude Code runs hooks as the user, with the hook's JSON on stdin and a JSON reply on stdout. Glassbox installs at most two.

Stop (every time Claude Code finishes a turn). Reads the transcript named in the hook input, runs the rules, and returns a one-screen summary (turns, tool calls, context peak, cost, top findings) that Claude Code shows in the terminal. Timeout 60 s. Always exits 0 — it never blocks a session because it failed. With **feedback** on (off by default), when a finding at or above the threshold has not been shown before in that session, the hook returns `decision: "block"` once with the findings as the reason; Claude reads them, answers in a sentence what it would do differently, then stops. That extra turn costs tokens (one short reply) and happens at most once per new finding per session.

SessionStart (only installed with `--context / plugin option context`; off by default). If `<project>/ .glassbox/last-session.md` exists, its text is returned as `additionalContext`, so the next session in that project starts knowing what went wrong last time. Timeout 15 s.

Two things to know about **context** before turning it on for a team:

1. It puts a file in the project directory whose contents are fed to the next session's model. The file is written by the hook from the transcript, and it is git-ignored, but anyone who can write to that directory can change what the next session is told. Treat `.glassbox/` like any other file that shapes agent behaviour (`CLAUDE.md`, `.claude/settings.json`).

2. The notes quote from the session (tool inputs and outputs, up to 6,000 chars). On a shared project directory that is data at rest.

How the CLI hook is invoked. `glassbox hook install` writes the command `npx -y glassbox-trace hook ...` into `settings.json`, so each run resolves the package through npm (cached after the first). To pin: `npm i -g glassbox-trace@0.6.2`, then `glassbox hook install --command glassbox ...`, or use the plugin, whose hooks run `node ${CLAUDE_PLUGIN_ROOT}/hooks/glassbox-hook.mjs` — the bundled copy, no npx.

There is no PreToolUse hook: Glassbox never approves, denies or alters a tool call.

7. What leaves the machine when you share a report

Nothing leaves unless a person sends it. The formats built for sending are:

- `check --redact --format json|md` — every string from the transcript is dropped (prompts, commands, results, titles, paths); what remains is tool names, counts, timings, token figures and cost. Where a finding would quote something it says «240 chars».
- `check --redact --legend F` — the same, except file paths become keys (`file:1a2b3c4d`, the first 8 hex of an HMAC-SHA256 with a random salt), so the report can say "this one file was read 48 times, 6 failed" without naming it. The key → path legend is written to `F` on the machine that ran the command and is not part of the report. `glassbox reveal report.md --legend F` reverses it, locally.
- The viewer's **Export redacted** — a structure-only `.jsonl` (timestamps, usage, tool names; no text).

Anything else (`open --out`, a plain `check` report, a transcript) contains session text and should be treated as such.

8. Cost figures

Costs are estimates from a rate card of Anthropic list prices (README, "Token and cost accounting"), or the `total_cost_usd` an Agent SDK run reports. Teams on negotiated rates, seats or credits pass their own card with `--rates FILE` or `GLASSBOX_RATES`; unknown models show "—" rather than a wrong number. The token counts are exact — they are read from the transcript.

9. Removal

- CLI hook: `glassbox hook uninstall` (removes the entries it added; `settings.json.glassbox-backup` is left for you to delete).
- Plugin: `/plugin uninstall glassbox@glassbox-trace`.
- Package: `npm uninstall -g glassbox-trace`; `npx` caches live under npm's cache directory (`npm cache clean --force` clears them).
- Files: `<temp>/glassbox-*.html`, `<temp>/glassbox-hook/`, any `.glassbox/` directory in a project, any legend or `--out` file you created.

Nothing else was changed on the machine.

10. Provenance and verification

- Source: github.com/aldohushi1-stack/glassbox, MIT. The engine is one file, `src/trace-core.js`, with no DOM and no imports; the CLI is `src/cli.mjs` and `src/tail.mjs`. It is small enough to read.
- npm releases are published by the repository's GitHub Actions workflow with `npm publish --provenance`, so npm shows which commit and workflow built each version.
- `npm test` runs the unit suite (parser, every rule, cost, redaction, legend, CLI, hooks, offline check); `npm run e2e` runs the browser suite with the request log; `npm run audit` runs the accessibility gate. All three run in CI on every push.
- To check a build yourself: open `dist/glassbox.html` with the browser's network panel open. It should show one entry, the file.

Questions: hi@aldo.ltd · Aldo Hushi, BlueprintAU, Adelaide.